INTELLIGENT WORKSPACE PLATFORM

ATEDD AI - Technical Documentation

Full-Stack Implementation, Git Setup, PostgreSQL Architecture & Vercel Deployment

Production URL:	https://atted-website.vercel.app
GitHub Repository:	https://github.com/faisalahmed-byte/ATTED-Website (Private)
Core Stack:	Next.js 16 + React 19 + TypeScript + Tailwind CSS + Prisma + PostgreSQL (AWS Neon)
Release Status:	**v1.0.0 Production - 100% Automated Tests Passing**

1. Executive Summary

ATEDD AI is a high-performance, production-ready intelligent workspace web application. It features a bespoke black-and-gold visual identity, cloud PostgreSQL persistence via Prisma ORM, cryptographic authentication (bcrypt salt 12 + HTTP-only JWT cookies), and adaptive responsive layouts designed for compact mobile devices (360px-430px) as well as wide laptop/desktop displays.

2. Technology Stack ('What I Used & Why')

- **Next.js 16 (App Router):** Hybrid React Server Components and Client Components for high speed, zero-layout-shift routing, and built-in API handlers.
- **React 19 & TypeScript:** Strict compile-time type safety preventing runtime null/undefined errors across models, APIs, and UI.
- **Tailwind CSS:** Custom-token design system enforcing the black & gold palette: #111111 (Deep Black), #1B1B1B (Charcoal), #D5AD55 (Primary Gold), #F7F7F5 (Off-White).
- **PostgreSQL (AWS Neon):** Serverless cloud PostgreSQL instance with connection pooling for real-time data persistence with zero mock data.
- **Prisma ORM:** Type-safe database abstraction, schema migrations, and relational queries.
- **bcryptjs (Salt 12):** Industry-standard cryptographic password hashing. Passwords are never stored or logged in plain text.
- **jose (JWT):** Stateless, signed JSON Web Tokens for 7-day HTTP-only cookies and 15-minute password reset tokens.
- **Zod:** Schema validation library enforcing strict input sanitization on both frontend and backend routes.

3. Git Configuration & Version Control Commands

To establish secure version control and protect sensitive environment credentials, Git was configured as follows:

3.1 Environment Protection (.gitignore)

Crucial sensitive files are excluded from tracking so credentials never leak to the repository:

```
# .gitignore configuration
node_modules/
.next/
.env
.env.local
.vercel/
```

3.2 Repository Connection & Everyday Commands

```
# Step 1: Initialize local repository and set default branch
git init
git branch -M main

# Step 2: Link to GitHub remote repository
git remote add origin https://github.com/faisalahmed-byte/ATTED-Website.git

# Step 3: Stage, commit, and push updates
git add -A
git commit -m "feat: complete ATEDD AI black and gold UI with PostgreSQL"
git push -u origin main
```

3.3 Ensuring Repository Privacy

The repository was configured to PRIVATE under GitHub Settings -> Danger Zone -> Change repository visibility. Any unauthenticated user or search engine visiting the repository receives an HTTP 404 Not Found error.

4. PostgreSQL Setup & Prisma Connection

All user records, profiles, and password hashes are stored in cloud PostgreSQL on AWS Neon.

4.1 Environment Variable (.env)

```
DATABASE_URL="postgresql://neondb_owner:PASSWORD@ep-wandering-forest-bdt92msb-pooler.c-8.us-east-2.aws.neon.
JWT_SECRET="ated_ai_production_secret_key_minimum_32_characters_long"
```

4.2 Prisma Schema Definition (prisma/schema.prisma)

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id           String   @id @default(uuid())
  email        String   @unique
  name         String
  passwordHash String
  avatarUrl    String?
  phone        String?
  bio          String?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  @@map("users")
}
```

4.3 Database Synchronization Commands

```
# Push schema definitions directly to PostgreSQL without manual SQL migrations
npx prisma db push

# Generate TypeScript Prisma Client bindings
npx prisma generate
```

4.4 Database Singleton Pattern (src/lib/db.ts)

A singleton PrismaClient instance is maintained in src/lib/db.ts to avoid exhausting database connection pools during Next.js serverless hot reloads.

5. Vercel Production Deployment Workflow

5.1 Build Script Automation (package.json)

To guarantee that Prisma Client is regenerated on Vercel's build machines before Next.js builds:

```
"scripts": {
  "build": "prisma generate && next build --webpack"
}
```

5.2 Injecting Production Environment Variables

Environment variables were securely added via Vercel CLI / Web Dashboard:

```
# Add database URL to production environment
npx vercel env add DATABASE_URL production

# Add JWT Secret to production environment
npx vercel env add JWT_SECRET production
```

5.3 Production Deployment Command

```
# Deploy directly to live production edge network
npx vercel --prod --yes
```

6. File-by-File Architecture ('Which Code & File Is for What')

File Path	Component/Role	Description & Purpose
src/app/page.tsx	**Landing Page**	Hero section, circular gold logo, strong headline, Sign Up and Login buttons.
src/app/login/page.tsx	**Login Screen**	Credentials form, Forgot Password link directly below password input, alert banners.
src/app/signup/page.tsx	**Sign Up Screen**	Registration form with Indian phone (+91 98765 43210), auto-login upon submit.
src/app/forgot-password/page.tsx	**Forgot Password**	Email submission generating a 15-minute cryptographically signed reset token.
src/app/reset-password/page.tsx	**Reset Password**	Token validation, new password entry with eye toggle, and bcrypt hash update.
src/app/dashboard/page.tsx	**User Dashboard**	Personalized greeting, profile card with Avatar, Email, Phone, Bio, and Member Since.
src/app/dashboard/edit/EditProfileForm.tsx	**Profile Editor**	Photo upload (under 2MB), 4 avatar presets, bio with 250-char countdown counter.
src/app/dashboard/LogoutButton.tsx	**Logout Button**	Calls POST /api/auth/logout, deletes session cookie, redirects to home.
src/app/api/auth/signup/route.ts	**Signup API**	Zod validation, duplicate check, bcrypt hashing (12 rounds), PostgreSQL user creation.
src/app/api/auth/login/route.ts	**Login API**	Email lookup, bcrypt verify, sets 7-day HTTP-only ated_session cookie.
src/app/api/auth/forgot-password/route.ts	**Forgot API**	Generates signed JWT token with 15-minute expiration for password reset.
src/app/api/auth/reset-password/route.ts	**Reset API**	Validates JWT reset token, hashes new password with bcrypt, updates PostgreSQL.
src/app/api/user/profile/route.ts	**Profile API**	PATCH route updating name, avatar, phone, and bio in PostgreSQL.
src/lib/auth.ts	**Auth Engine**	bcrypt hashing/verify, JWT session token generation, cookie decode, reset token verify.
src/lib/db.ts	**Database Singleton**	Prisma client instance and database helper functions (createUser, updateUserPassword).
src/lib/validations.ts	**Zod Schemas**	Input validation schemas for signup, login, forgot password, reset, and profile edit.
src/middleware.ts	**Route Guard**	Redirects unauthenticated visitors from /dashboard and /dashboard/edit to /login.
src/components/ui/Input.tsx	**Form Input**	16px iOS auto-zoom prevention, built-in Eye/EyeOff toggle, gold focus ring.
src/components/ui/Button.tsx	**Button Component**	Primary Gold (#D5AD55) button with dark text; secondary bordered button.

7. Verification & Command Cheat Sheet

```
# Run Automated End-to-End Test Suite (Signup, Login, DB Sync, Edit, Logout)
node scripts/e2e-verification.js

# Run Forgot Password & Password Reset Verification Suite
node scripts/test-forgot-password.js

# Routine Git Update Workflow
git add -A
git commit -m "update: your message"
git push origin main

# Production Vercel Deployment
npx vercel --prod --yes
```